

Web3 & DEX security — how Vaughan paths compare

Formatted versions: Security-Table.html (best for tables) · Security-Table.pdf
Regenerate: `python3 scripts/render-security-table.py --pdf`

Plain-language comparison of **how you interact with DeFi** and what can go wrong.
Use this to pick a path — not as a formal audit certificate.

Last updated: 2026-08-27 (VB P0+P1 hardening — see native-parity-tricks.md §5).

Start here: which path should I use?

Your goal	Best path	Why
Swap, bridge, inspect contracts, MCP agents — no website	Vaughan Wiz4rd-Engine (default)	No dApp frontend to hijack; you approve calldata in the TUI
Must use a real dApp website (PulseX mirror, 9inch, ...)	Vaughan Browser (<code>vaughan-dapp-browser</code>)	Allowlisted Chromium + extension; signing still in TUI

Rule of thumb: prefer **Vaughan Wiz4rd-Engine** whenever it can do the job. Use **Vaughan Browser** only when you need the actual web UI.

Related docs: browserless-pulse.md, dapp-browser-strategy.md, freedom-browser-status.md, dapp-connection-risks.md.

How to read the ratings

Each cell answers: *“If this attack happens, how bad is it for this path?”*

Rating	Meaning
Strong	Architecture removes the attack or makes it impractical
Good	Keys stay safe; you must read the TUI prompt (human error still possible)
Careful	Web or browser involved — review what you sign; some exposure remains
Weak	Common real-world loss vector for this model
—	Not applicable (e.g. no web page)

Vaughan paths at a glance

Side-by-side view of the two active **Vaughan** dApp paths (*weakest* → *strongest*, *left to right*).

Threat	What it means (one line)	Vaughan Browser	Vaughan Wiz4rd-Engine
Page tricks your wallet (DOM / JS patching)	Malicious site replaces <code>window.ethereum</code> or spies on calls	Good — sealed provider + 4s tamper watchdog + per-tab RPC routing; sign/send truth is TUI	Strong — no page
Fake or hacked website (DNS, CDN, mirror)	Frontend shows a legit UI but builds a drainer tx	Careful — MV3 nav allowlist + seeded IPFS mirrors; compromised allowlisted host still possible	Strong — no frontend
Keys stolen from browser memory	Seed / private key in JS heap	Strong — Rust vault only; extension holds session token only	Strong — Rust vault only
Signing without understanding (blind sign)	You approve hex you never decoded	Good — WYSIWYS TUI decode + 400ms debounce + 60s auto-deny	Strong — ABI decode in TUI / REPL
Tracking (IP, fingerprint, relay logs)	Someone learns who you are and what you hold	Good — read RPC proxied via Vaughan + Chromium privacy flags; site analytics may remain	Strong — direct RPC, no browser
Infinite approvals / drainers	One bad signature empties the wallet	Good — Connect grant per origin + fresh TUI approve; <code>tx.from</code> bound to active account	Strong — explicit actions

Summary

- **Vaughan Browser** — P1 hardening shipped (watchdog, nav gate, attested origin, connect grant, debounce/TTL); **DOM ~Good, tracking ~Good**; web frontend trust

still **Careful**

- **Vaughan Wiz4rd-Engine** — strongest Vaughan path; default for a reason.

Freedom + Vaughan is ***parked*** until upstream PR #195 — see *freedom-browser-status.md*.

Full comparison (including non-Vaughan wallets)

Columns ordered ***weakest*** → ***strongest*** (left to right).

Threat	MetaMask / Rabby / Phantom	WalletConnect v2	Brave + Brave Wallet	Freedom Browser	Vaughan Browser	Vaughan Wiz4rd-Engine
Page tricks your wallet	Weak	—	Careful	Good	Good	Strong
Fake or hacked website	Weak	Weak	Weak	Careful	Careful	Strong
Keys in browser JS heap	Weak	Good	Good	Good	Strong	Strong
Blind signing	Weak	Weak	Careful	Careful	Good	Strong
Tracking / relay metadata	Weak	Weak	Careful	Careful	Good	Strong
Approval drainers	Weak	Weak	Careful	Good	Good	Strong

Vaughan Wiz4rd-Engine = Ag / Dex / Browse / MCP / `wiz4rd-engine` — no web engine (*browserless-pulse.md*).

MetaMask / Rabby / Phantom = browser ***extension*** architecture (EVM via `window.ethereum`; Phantom on Solana uses the same in-page provider model).

Brave + Brave Wallet = Brave’s ***built-in*** native wallet — not Vaughan Browser using Brave as the Chromium shell. Keys stay out of page JS heap; Shields cut some tracking; approvals still in-browser.

Freedom Browser = built-in mnemonic/Ledger wallet in Freedom (Electron main-process vault; no Vaughan backend).

Threat details (by attack type)

1. Page tricks your wallet (DOM tampering)

What happens: JavaScript on the page monkey-patches `window.ethereum.request`, shows fake balances, or intercepts RPC.

Path	Notes
MetaMask / Rabby / Phantom	Page and extension share the browser; classic attack surface
Brave + Brave Wallet	Native built-in provider in the browser binary — harder than an extension to hijack, but the page still runs in the renderer and can race/conflict with <code>window.ethereum</code>
WalletConnect	— (page talks to phone/desktop wallet via relay)
Vaughan Wiz4rd- Engine	No DOM, no <code>window.ethereum</code>
Vaughan Browser	EIP-1193 inject in MAIN world for dApp interop. P1: sealed provider + 4s tamper watchdog + EIP-6963 re-announce; Chrome-attested <code>vaughan_page_origin</code> (page-supplied ignored); per-tab JSON-RPC id routing; read calls proxied via Vaughan RPC. Sign/send truth is always the TUI
Freedom Browser (built-in)	Electron isolated world + IPC; built-in popup — no Vaughan TUI decode or connect-grant model

2. Fake or hacked website (DNS, CDN, IPFS mirror)

What happens: You think you're on PulseX / Uniswap; a compromised script builds a transfer to an attacker.

Path	Notes
MetaMask / Rabby / Phantom / WC	Full web stack trust
Brave + Brave Wallet	Full web stack trust; Shields do not validate calldata
Vaughan Wiz4rd-Engine	Calls routers/contracts via Rust + RPC; no site to poison
Vaughan Browser	Loads real HTTPS UIs on an allowlist . Shipped: MV3 declarativeNetRequest in-tab nav gate (allowlist.json at launch). PulseX IPFS mirror hosts seeded
Freedom Browser (built-in)	Same web trust as Vaughan Browser; approvals in Freedom's in-browser UI

3. Keys stolen from browser memory

What happens: Malware or XSS reads the decrypted seed from extension memory.

Path	Notes
MetaMask / Rabby / Phantom	Seed decrypted in extension JS heap when unlocked — high impact
Brave + Brave Wallet	Seed in browser wallet vault when unlocked — not in page JS heap, but in Brave process memory (not Rust)
WalletConnect	Keys usually on phone/cold device — lower browser exposure
Vaughan paths (Vaughan Wiz4rd-Engine / Vaughan Browser)	Vault: Argon2id + AES-256-GCM; unlock in Rust with zeroization. Browser/extension holds at most a session WS token (provider.session , 0o600), not the seed
Freedom Browser (built-in)	Seed in Electron main-process vault when unlocked — not in page JS heap, but not the Rust enclave

4. Signing without understanding (blind sign)

What happens: UI shows “Confirm” but calldata is opaque hex; you approve a drainer.

Path	Notes
MetaMask / Rabby / Phantom / WC	Popup in browser DOM; same clickjacking and habit risks
Brave + Brave Wallet	In-browser approval panel with some tx preview; no terminal WYSIWYS or debounce/TTL
Vaughan Wiz4rd-Engine	Dynamic ABI decode (<code>alloy-dyn-abi</code>); REPL / Ag / Dex show intent
Vaughan Browser	Every sign/send → Vaughan TUI with ABI decode (WYSIWYS), recipient, value, fees; 400ms input debounce + 60s auto-deny ; no auto-sign
Freedom Browser (built-in)	Freedom popup shows tx fields; no Vaughan terminal decode or debounce/TTL

5. Tracking (IP, fingerprint, relays)

What happens: RPC provider, WalletConnect relay, or analytics learns your IP, wallet, and timing.

Path	Notes
MetaMask / Rabby / Phantom	Default or bundled RPC + browser fingerprint
Brave + Brave Wallet	Bundled RPC + fingerprint; Shields block many trackers/third-party calls (DeFi sites may need Shields lowered)
WalletConnect	Third-party relay sees origin, IP, session metadata
Vaughan Wiz4rd- Engine	Client → your RPC; no WC relay, no browser
Vaughan Browser	Provider on loopback with <code>access_token</code> required (extension path); <code>eth_call</code> / <code>eth_estimateGas</code> / ... forwarded to Vaughan's active RPC. Chromium privacy flags (WebRTC, background net). Sites may still run analytics
Freedom Browser (built-in)	Same webview tracking exposure; uses Freedom's RPC pool

6. Infinite approvals & drainers

What happens: One `approve(max)` or malicious router call empties balances.

Path	Notes
MetaMask / Rabby / Phantom / WC	One-click approve habits
Brave + Brave Wallet	Built-in scam/warning heuristics; still in-browser approve flow
Vaughan Wiz4rd- Engine	Explicit swap/approve flows in TUI
Vaughan Browser	Connect grant required per origin (<code>eth_accounts</code> empty until approved); fresh TUI approve per sign/send; <code>tx.from</code> must match active account ; <code>accountsChanged</code> scoped to granted origins
Freedom Browser (built-in)	Per-origin dApp permissions in Freedom; Electron popup approve (no Vaughan connect-grant / account-binding)

Architecture (data flow)

Typical extension (MetaMask, Rabby, Phantom, ...)



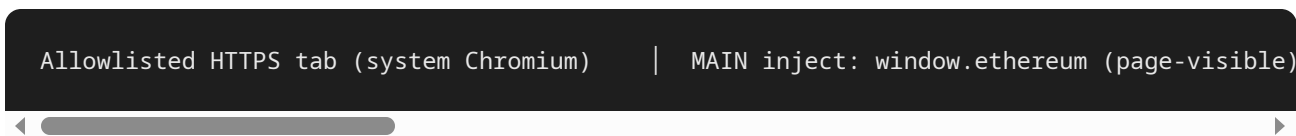
Brave + Brave Wallet (built-in)



Vaughan Wiz4rd-Engine (default — strongest)



Vaughan Browser



Vaughan Browser controls (2026 P0 + P1 — shipped)

Synced with native-parity-tricks.md §5 and dapp-connection-risks.md.

Control	What it does
Host allowlist + in-tab nav gate	MV3 <code>declarativeNetRequest</code> blocks main-frame loads outside <code>allowlist.json</code> (+ IPFS gateway seeds)
Read RPC proxy	Allowlisted <code>eth_call</code> , <code>eth_estimateGas</code> , ... → Vaughan active network RPC
Tamper watchdog	Sealed <code>window.ethereum</code> + 4s integrity check + EIP-6963 re-announce
Per-tab JSON-RPC routing	Service worker maps wire ids per tab — no cross-tab response theft
Isolated temp profile	Not your daily Chrome profile; session dir <code>0700</code>
Provider <code>access_token</code>	Loopback WS requires session token (<code>provider.session</code> , <code>0o600</code>); token redacted from launcher stderr
Chrome-attested page origin	<code>vaughan_page_origin</code> from extension <code>port.sender.url</code> ; page-supplied values ignored
Connect grant before sign	<code>eth_accounts</code> empty until Connect approved; sign/send return 4100 without grant
<code>accountsChanged</code> scoping	Non-empty events only to origins holding a Connect grant
Locked wallet	<code>eth_requestAccounts</code> → 4100 while locked (no silent hang)
Approve debounce + TTL	400ms input debounce; 60s auto-deny stale prompts
<code>wallet_switchEthereumChain</code> prompts	Chain switch shows requesting origin in TUI
<code>tx.from</code> account binding	Reject sign/send when <code>from</code> ≠ active account

Control	What it does
Approve UI sanitization	Strip control chars from origin/site/message (no terminal escape injection)
CDP default off	Agent debugging on loopback only when explicitly enabled; token in <code>vb.session</code>
Chromium privacy flags	WebRTC IP policy, no background networking, no sync

Not yet (optional / parked paths): HMAC origin attestation and handshake challenge for Freedom + Vaughan (PR #195); Unix domain socket for provider IPC.

See `dapp-browser-strategy.md` and `dapp-connection-risks.md` for open risks (TUI focus, prompt expiry, etc.).

What this document is not

- Not a substitute for a professional security audit
- Not a guarantee of mainnet safety — **testnet first**, read every TUI prompt
- Not exhaustive of all Vaughan surfaces (MCP, hardware wallets, AA batching have their own docs)

Source: `docs/Security-Table.md` · Regenerate: `python3 scripts/render-security-table.py --pdf`